HyperEuclidean MIDI Generator

User Manual

1. Overview

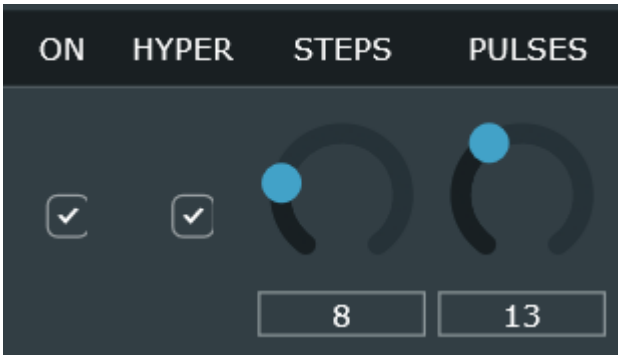
HyperEuclidean MIDI Generator is a 6-track algorithmic sequencer based on Euclidean rhythm geometry. Instead of traditional step-by-step programming, you define the "density" of notes within a time space, allowing for the creation of complex polyrhythms and generative melodies in seconds.

2. Track Controls (The Channel Strip)

Each of the 6 rows functions as an independent generator. Moving from left to right, the user can configure:

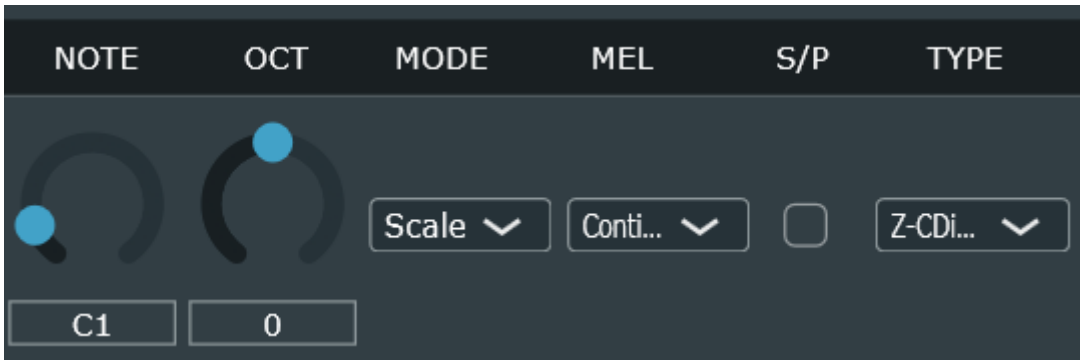
A. Core Logic & Activation

- **ON (Toggle):** Enables or disables MIDI generation for the specific row.
- **HYPER (Toggle):** * *Off:* Uses standard Euclidean distribution (Bjorklund algorithm).
    - *On:* Activates the **Hyper-Euclidean** mode, distributing pulses across multiple "depth" layers for irregular, organic patterns.



B. Rhythmic Shaping

- **STEPS (Slider 1-64):** The total length of the sequence cycle.
- **PULSES (Slider 1-64):** The number of notes (beats) to be distributed within the Steps.
- **SWING (Slider 0-100%):** Offsets even-numbered steps to create a "groove" or "human feel."
- **PROB (Slider 0-100%):** The probability that a programmed pulse will actually play. **Choice:** Set to 100% for mathematical precision, or lower for evolving, unpredictable patterns.



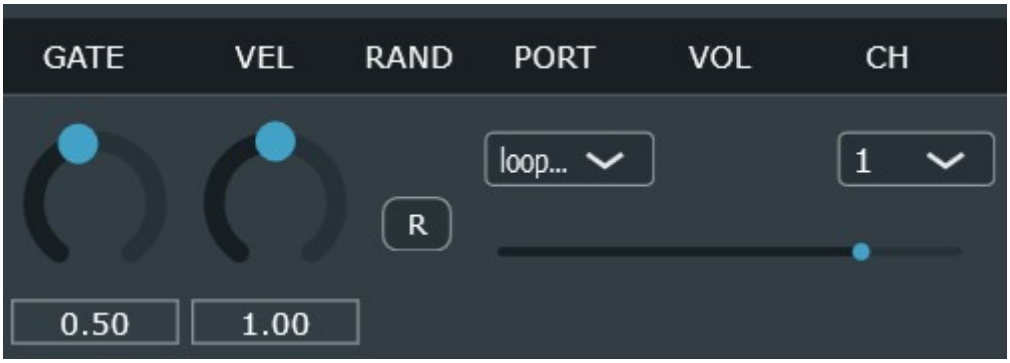
C. Melodic & Pitch Selection

- **NOTE (Slider/Value):** The root MIDI note (e.g., C3).
- **OCT (Slider -1 to +4):** Real-time octave shifting.
- **MODE (Dropdown Menu):**

    - **Fixed:** Always plays the root MIDI note.
    - **Input:** Follows external MIDI notes (MIDI Learn/Follow).
    - **Scale:** Generates notes based on a selected musical scale.
    - **Chord:** Generates notes based on specific chord intervals.
- **TYPE (Dropdown Menu):** Dynamically populates based on the **Mode**. **Choice:** Select from dozens of presets (e.g., *Z-DMixolydian*, *PentaMinor*, *Cmaj7-Jazz*, etc.).
- **MEL (Dropdown Menu):**
    - **Continuum:** The melodic index advances indefinitely regardless of rhythm length.
    - **Looped:** The melody resets its position whenever the Euclidean cycle restarts.
- **S/P (Step/Pulse Toggle):**
    - *Inactive:* Melody advances on every **Step** (silent steps still move the melody forward).
    - *Active:* Melody advances only on a **Pulse** (the sequence only moves when a note is triggered).

D. Performance & Output

- **GATE (Slider 0-100%):** Controls note duration. **Choice:** Low values for *staccato*, high values for *legato*.
- **VEL (Slider 0.0 - 2.0):** Velocity multiplier. It scales the internal velocity (base 80 / accent 120) to match your synth's sensitivity.
- **RAND (Button):** Instantly randomizes all parameters for the row—ideal for "happy accidents."
- **PORT (Standalone only):** Selects the physical or virtual MIDI output (e.g., *loopMIDI*, *USB MIDI Interface*).
- **VOL (Slider 0-127):** Sends MIDI CC7 (Main Volume) to the receiving instrument.
- **CH (Dropdown 1-16):** Sets the specific MIDI channel for the row.



3. Global Controls (Footer)

- **PLAY/STOP:** Starts/Stops the internal sequencer (Standalone mode).
- **BPM:** Sets the tempo (40 - 240 BPM).
- **EXT MIDI CLOCK (Toggle):** * *Off:* Uses the internal BPM slider.
    - *On:* Syncs automatically to your DAW (VST3) or external hardware clock.

---

4. Technical Specifications

Software Architecture

- **Framework:** Built with **JUCE 8 (C++)**.
- **Parameter Management:** Utilizes AudioProcessorValueTreeState for thread-safe communication and full DAW automation support.
- **Timing:** Sample-accurate MIDI triggering processed within the processBlock loop, ensuring zero jitter even at high BPMs.

Mathematical Core

The engine uses a modified version of the **Bjorklund Algorithm**, originally designed for neutron accelerators and later adapted for ethnomusicology. The algorithm ensures that k pulses are distributed as evenly as possible among n steps, creating the "maximum evenness" characteristic of traditional world rhythms.

UI Rendering

The interface uses **Direct2D** (on Windows) for hardware-accelerated graphics. Custom textFromValueFunction lambdas are used to translate raw MIDI data into human-readable musical notation (e.g., Note 60 $\rightarrow$ "C3") in real-time.